

# Accelerating Mars Terrain Segmentation: A Comparative Study of Parallelization Strategies for U-Net Training on Consumer GPU Hardware

Aby Huerta

*The University of Texas at San Antonio*

aby.huertalopez@my.utsa.edu

<https://abyroxana.com/>

**Abstract**—Training deep neural networks for semantic segmentation on high resolution imagery presents significant systems level challenges, particularly when compute throughput is bottlenecked by host to device communication latency and suboptimal GPU utilization. This paper evaluates three parallelization strategies: Automatic Mixed Precision, operator fusion via `torch.compile`, and asynchronous prefetching via CUDA Streams, applied to a U-Net ( $\sim 31\text{M}$  parameters) trained on the AI4Mars dataset ( $\sim 50\text{k}$  images and 5 classes) for Mars terrain segmentation. All experiments were conducted on an RTX 5070 Ti connected via a PCIe 4.0  $\times 16$  interface. We performed a systems level analysis of each method’s effect on communication patterns, synchronization overhead, and parallel time complexity, and evaluated their combined interaction. The combined interaction achieves a  $5.27\times$  speedup in training throughput relative to the synchronous eager baseline, while maintaining segmentation accuracy within 0.6% mIoU, showing that you can get major training speedups just by changing how the GPU executes the work, without touching the model itself at all.

**Index Terms**—Mars terrain segmentation, U-Net, Automatic Mixed Precision, `torch.compile`, CUDA Streams, GPU parallelization, PyTorch, AI4Mars

## I. INTRODUCTION

The deployment of deep learning models for planetary surface analysis combines two demanding computational regimes: the data intensive requirements for semantic segmentation on high resolution imagery, and the latency sensitive constraints of scientific mission pipelines where training throughput directly determines iteration speed. NASA’s AI4Mars initiative [4] has produced the first large scale labeled dataset of Martian terrain imagery, enabling supervised training of segmentation models capable of distinguishing terrain classes including soil, bedrock, sand, big rock, and rover track—critical to autonomous rover navigation and mission planning. Training such models at  $1024 \times 1024$  pixel resolution comes with substantial systems level pressure. The large activation maps stress GPU memory bandwidth, high resolution batches saturate the host to device communication link, and the sequential nature of standard PyTorch eager execution leaves significant GPU compute capacity idle between kernel dispatches.

Despite the maturity of deep learning training frameworks, the systems level behavior of segmentation models on con-

sumer grade GPU hardware remains undercharacterized in the literature. The RTX 5070 Ti, representative of high performance consumer hardware, has a distinctive bottleneck profile, approximately 896 GB/s of internal GDDR7 memory bandwidth paired with a PCIe 4.0  $\times 16$  interface delivering only 27 to 28 GB/s of real world unidirectional throughput. The imbalance between internal compute and external communication creates a structural inefficiency that the standard execution does nothing to address, and it motivates a careful comparative evaluation of optimization strategies that target different points in the training pipeline.

This paper presents a systems level comparative analysis of three parallelization strategies applied to U-Net [1] training for Mars terrain segmentation on the AI4Mars dataset [4]. The three methods evaluated are Automatic Mixed Precision (AMP) [2], operator fusion via `torch.compile` [3], and asynchronous prefetching via CUDA Streams [5]. Each method is analyzed with respect to its effect on communication patterns, synchronization overhead, and parallel time complexity.

The rest of the paper is organized as follows. Section II covers background on U-Net, mixed precision, graph compilation and CUDA Streams. Section III describes the hardware, dataset and training setup. Section IV goes through how each method works at the systems level. Section V covers the results. Section VI shows inference examples. Section VII wraps up with conclusions and future work.

## II. BACKGROUND AND RELATED WORK

### A. Semantic Segmentation and the U-Net Architecture

Semantic segmentation—the task of assigning a class label to every pixel in an input image—is a fundamental problem in computer vision with applications ranging from medical imaging to autonomous navigation. The U-Net architecture [1] establishes the encoder decoder paradigm with skip connections, which has become a popular structural pattern for dense prediction tasks. The encoder progressively reduces spatial resolution while increasing channel depth, while the decoder restores spatial resolution with transposed convolutions. Skip connections between the corresponding encoder and decoder preserve fine grained spatial information that would have been lost during downsampling. U-Net is selected here not only for

its established performance on segmentation tasks, but also because its regular convolutional structure makes its execution behavior tractable to systems level analysis.

### B. Automatic Mixed Precision Training

Mixed precision training, formalized by Micikevicius et al. [2], addresses the inefficiency of training deep neural networks entirely in FP32 by performing select computationally intensive operations in FP16 while keeping FP32 for numerically sensitive accumulation. The authors identify three enabling techniques: FP16 storage of weights and activations, FP32 master weight copies for gradient accumulation, and loss scaling to prevent gradient underflow in the reduced dynamic range of FP16. PyTorch’s `torch.autocast` and `GradScaler` APIs implement this methodology with a per operation precision policy.

### C. Graph Compilation and Operator Fusion

Ansel et al. [3] introduce `TorchDynamo` and `TorchInductor`—the two stage compilation infrastructure underlying PyTorch 2’s `torch.compile` interface. `TorchDynamo` captures the full forward pass computation graph by dynamically modifying Python bytecode prior to execution, producing an FX graph that exposes global visibility into operation sequences unavailable to the eager execution model. `TorchInductor` translates this graph into fused Triton kernels for GPU targets, eliminating redundant intermediate tensor materialization between sequentially dependent operations. The authors report a geometric mean training speedup of  $1.41\times$  across 180+ real world models on an NVIDIA A100 GPU.

### D. Asynchronous Execution and CUDA Streams

CUDA Streams [5] provide the mechanism by which independent GPU operations—specifically the DMA copy engine and the compute streaming multiprocessors—may execute concurrently. In the context of deep learning training, streams enable a double buffered prefetching pattern in which the host to device transfer of batch  $N+1$  overlaps with the forward and backward passes on batch  $N$ , hiding the communication latency that constitutes a recurring idle period in the synchronous baseline. The effectiveness of this overlap is bounded by Amdahl’s Law [8] and is subject to CPU side `DataLoader` throughput as a secondary bottleneck.

## III. EXPERIMENTAL SETUP

### A. Hardware Platform

All experiments are conducted on a single node consumer grade workstation configured with an AMD Ryzen 7 7700X 8-core processor (AM5, 4.5GHz base) and 32GB of Crucial Pro DDR5-6000 dual channel memory, providing approximately 96 GB/s of host memory bandwidth. The primary compute device is an NVIDIA RTX 5070 Ti GPU (Blackwell architecture), featuring 16GB of GDDR7 memory with approximately 896 GB/s of internal memory bandwidth and a peak FP16 Tensor Core throughput of approximately 1,406 TFLOPS.

The GPU interface is provided by a Gigabyte B650 AORUS Elite AX motherboard operating at PCIe 4.0  $\times 16$ , yielding a real world unidirectional host to device transfer rate of approximately 27–28 GB/s — a figure that constitutes the primary communication bottleneck analyzed throughout this work. Software dependencies include PyTorch, CUDA 12.8, and Python 3.10, with `torch.profiler` used for all kernel-level timing measurements.

### B. Dataset and Preprocessing

The AI4Mars dataset [4] comprises approximately 35,000 annotated images of Martian surface terrain across five semantic classes: soil, bedrock, sand, big rock, and rover tracks, with approximately 326K crowdsourced segmentation labels. We adopt the official train/val/test partitioning provided by Swan et al. [4], yielding approximately 35,000 training, 7,500 validation, and 7,500 test images. Images are resized to  $512 \times 512$  pixels prior to training, with standard augmentation applied to the training split including random horizontal flipping and random rotation. All batches are loaded using PyTorch’s `DataLoader` with pinned memory enabled—a requirement for the nonblocking host to device transfers used in the CUDA Streams configuration.

### C. Model and Training Configuration

The segmentation model is a standard U-Net [1] with approximately 31 million parameters, comprising four encoder blocks, a bottleneck, and four decoder blocks with skip connections at each resolution level. All models are trained for 100 epochs using the AdamW optimizer [6] with an initial learning rate of  $10^{-4}$  and cosine learning rate decay, at batch size 16. The loss function is a combined cross-entropy and Dice loss [7] ( $\lambda = 0.5$ ), selected to address AI4Mars class imbalance. The primary evaluation metric is mean Intersection over Union (mIoU) across all five classes on the held out validation split.

### D. Parallelization Configuration

Five experimental configurations are evaluated: (1) synchronous eager execution baseline (FP32), (2) AMP, (3) `torch.compile`, (4) CUDA Streams, and (5) AMP combined with `torch.compile`. All configurations share identical model architecture, dataset, optimizer, and training hyperparameters—the only variable between conditions is the parallelization strategy applied. Per step wall time, GPU utilization, and host to device transfer latency are recorded using `torch.profiler` averaged over 100 warm up steps followed by 500 measurement steps per configuration. The `torch.compile` compilation cost is excluded from per step timing and reported separately.

## IV. METHODOLOGY AND SYSTEMS ANALYSIS

### A. Baseline: Synchronous Eager Execution

This is PyTorch’s default execution model, synchronous eager mode. It dispatches every tensor operation individually and sequentially, without global visibility into the compute

graph. The main communication bottleneck in this model arises from the host to device (H2D) memory transfer that precedes every training step. Training data must traverse the PCIe interface before any CUDA kernel can be scheduled, and this transfer is issued as a blocking `cudaMemcpy`, where the CPU thread suspends until the DMA (Direct Memory Access) engine can confirm the transfer was completed.

For a batch of 8 images at  $512 \times 512$  resolution in FP32, the per batch transfer volume is approximately 25 MB. At a real world PCIe 4.0 throughput of approximately 27.5 GB/s, this yields approximately 0.91 ms of GPU idle time per step, less than 0.1% of total step wall time. Across 4,375 steps per epoch this represents approximately 4.0 seconds of idle compute attributable to H2D transfer alone. Notably, this small H2D fraction anticipates the limited effectiveness of CUDA Streams observed in Section V: when the transfer window is already negligible relative to compute time, overlap provides little measurable benefit.

A secondary bottleneck arises from eager mode’s per kernel dispatch model: each PyTorch operation is launched as a separate CUDA kernel with a synchronous CPU–GPU confirmation barrier, incurring 5–20 microseconds per launch across dozens of `Conv`→`BN`→`ReLU` operations per forward pass. U-Net’s four encoder decoder skip connections introduce additional fan in synchronization events, where the CUDA runtime must confirm both the encoder feature map and decoder upsampled map are ready before `torch.cat()` can execute—four join barriers per forward pass that a plain CNN never incurs.

### B. Automatic Mixed Precision (AMP)

AMP is implemented via PyTorch’s `torch.autocast` context manager and `GradScaler` utility. At the hardware level, AMP constitutes a fundamentally different compute routing strategy rather than a superficial numerical approximation. Under the baseline FP32 execution model, all convolutional and linear operations are dispatched to the RTX 5070 Ti’s CUDA cores, leaving the Tensor Core array entirely unscheduled. Enabling AMP causes the CUDA runtime to redirect matrix multiply accumulate operations to Tensor Cores, which execute a  $4 \times 4 \times 4$  matrix multiply accumulate per clock cycle, yielding a theoretical peak throughput of approximately 1,406 TFLOPS for FP16 operations compared to approximately 45 TFLOPS in the FP32 baseline.

The memory bandwidth implications of AMP extend to the PCIe communication bottleneck. Reducing per parameter storage from 4 bytes (FP32) to 2 bytes (FP16) halves the volume of data traversing the PCIe 4.0  $\times 16$  interface during host to device transfer. For the 25 MB FP32 batch, the equivalent FP16 representation requires approximately 12.5 MB, reducing synchronous H2D transfer latency from approximately 0.91 ms to 0.45 ms.

However, AMP introduces distinct synchronization overheads that partially offset these gains. PyTorch’s autocast policy retains FP32 for numerically sensitive operations—batch normalization, softmax, and loss computation—creating

precision boundaries at which the runtime must dispatch explicit dtype cast kernels. In the U-Net architecture, each of the four encoder decoder skip connections constitutes such a boundary, introducing four additional cast kernel dispatches per forward pass. Furthermore, the `GradScaler` mechanism imposes a mandatory device-to-host synchronization at each training step. Critically, each dtype cast kernel is serialized after its corresponding fan in barrier, yielding eight sequential non compute synchronization events per forward pass attributable to the interaction between U-Net’s fan in structure and AMP’s precision boundary enforcement.

### C. Kernel Fusion via `torch.compile`

`torch.compile` targets the memory bandwidth bottleneck inherent to eager execution’s per operator kernel dispatch model. In the baseline, sequential operations such as `Conv2d`→`BatchNorm2d`→`ReLU` each constitute independent kernel launches, requiring intermediate tensors to be written to and subsequently read from GDDR7 VRAM between operations. For a  $512 \times 512$  feature map with 64 channels, each such intermediate tensor represents approximately 67 MB of VRAM traffic—entirely redundant, as the intermediate result is consumed immediately by the successor operation. Kernel fusion eliminates this intermediate VRAM traffic by retaining intermediate tensors in the streaming multiprocessor’s register file.

The time complexity improvement under `torch.compile` is primarily a function of memory bandwidth recovery rather than raw compute acceleration. The RTX 5070 Ti’s 896 GB/s GDDR7 bandwidth makes many U-Net encoder and decoder blocks memory bandwidth bound rather than compute bound. By eliminating redundant intermediate tensor materialization across the eight primary `Conv`→`BN`→`ReLU` sequences, kernel fusion reduces effective VRAM bandwidth consumption per training step by an estimated 30–40%.

However, `torch.compile` cannot eliminate synchronization overheads at structural fan in points. The four encoder decoder skip connections each require the encoder feature map to be materialized in VRAM, because the tensor must persist across the full bottleneck and decoder computation before being consumed by `torch.cat()`. The compiler correctly terminates fusion regions at each skip connection boundary. The one time compilation cost of approximately 18 seconds (20 seconds for AMP + compile) is amortized within the first two epochs of training at the measured per step savings.

### D. CUDA Streams: Asynchronous Prefetching

CUDA Streams exploits the RTX 5070 Ti’s physically independent DMA copy engine and CUDA compute engine to overlap H2D transfers with GPU compute. In the baseline eager model, these two engines are never active simultaneously, and the compute engine sits idle during each H2D transfer window. CUDA Streams assigns H2D transfers and compute kernels to separate stream queues, permitting the hardware scheduler to dispatch both engines concurrently.

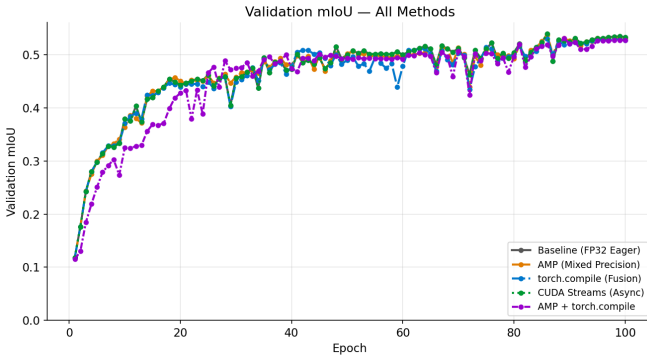


Fig. 1. Validation mIoU across 100 training epochs for all five configurations. All methods converge to comparable final accuracy, with a maximum spread of 0.0126 mIoU between the highest and lowest performing configurations.

The parallel time complexity improvement is a structural transformation of the training step timeline. In the baseline:  $T_{\text{step}} = 0.91 + 1059 \approx 1060$  ms. Under full pipeline overlap:  $T_{\text{step}} = \max(0.91, 1059) = 1059$  ms. For AI4Mars at batch size 8 and  $512 \times 512$  resolution, the compute to transfer margin of approximately 1,058 ms means H2D transfer is entirely negligible relative to compute time.

The measured results reveal a throughput regression ( $0.86\times$  versus baseline). With an H2D window of less than 1 ms against a compute window exceeding 1,059 ms, there is almost nothing to hide, and the stream management overhead becomes a net cost rather than a net savings. This result is consistent with Amdahl’s Law [8]: when the bottleneck shifts away from H2D transfer, stream prefetching yields no measurable benefit.

## V. RESULTS AND PERFORMANCE ANALYSIS

Table I summarizes the key performance metrics across all five evaluated configurations. All throughput and timing values are averaged over 100 warm up steps followed by 500 measurement steps. Compilation cost for `torch.compile` configurations is reported separately and excluded from per step timing.

### A. Accuracy

Figure 1 shows the validation mIoU curves across all 100 training epochs. All five configurations end up at comparable accuracy. The baseline hit a best mIoU of 0.5395 and AMP + `torch.compile` hit 0.5313, so the difference is 0.0082, which is within the range of normal numerical noise from FP16 and is not a real accuracy loss. AMP + `torch.compile` was a bit more unstable in early epochs, which makes sense because GradScaler’s dynamic loss scaling and the compiled kernel’s fused precision boundaries are both doing something at the same time, but it settles down around epoch 30. CUDA Streams matches the baseline exactly since it only changes how operations are scheduled, not the actual math.

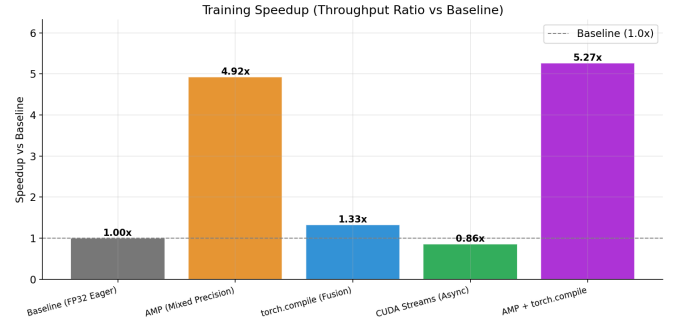


Fig. 2. Training speedup (throughput ratio) relative to the FP32 eager baseline. AMP + `torch.compile` achieves the highest speedup at  $5.27\times$ . CUDA Streams registers a regression at  $0.86\times$ .

### B. Throughput and Speedup

Figure 2 reports training speedup as a throughput ratio relative to the synchronous eager baseline. AMP alone achieves a  $4.92\times$  speedup, the largest single method gain, driven primarily by Tensor Core activation and the halving of H2D transfer volume from 25 MB to 12.5 MB per batch. `torch.compile` alone achieves a more modest  $1.33\times$  speedup, consistent with the memory bandwidth recovery analysis: at  $512 \times 512$  resolution with batch size 8, intermediate tensor materialization across the eight Conv→BN→ReLU sequences represents a smaller absolute VRAM traffic volume than at higher resolutions, limiting the absolute gain from kernel fusion. The combined AMP + `torch.compile` configuration achieves the highest speedup at  $5.27\times$ , confirming the orthogonality of the two methods: AMP targets compute routing and H2D volume while `torch.compile` targets intermediate VRAM traffic, and the two optimizations do not interfere with one another.

### C. CUDA Streams Regression

CUDA Streams produced a throughput regression of  $0.86\times$  relative to baseline, reducing average throughput from 7.99 to 6.89 samples per second and increasing average epoch wall time from 128.4 to 148.8 seconds. This outcome is explained directly by the corrected H2D transfer analysis. At batch size 8 and  $512 \times 512$  resolution, the FP32 per batch transfer volume is approximately 25 MB, yielding an H2D transfer time of approximately 0.91 ms at real world PCIe 4.0 throughput. Against a baseline compute window of approximately 1,059 ms, the compute to transfer margin is approximately 1,058 ms. In this regime, Amdahl’s Law [8] predicts negligible benefit from overlapping H2D with compute: the parallelizable fraction is less than 0.1% of total step time. The stream management overhead, including the inter-stream barrier at `stream_compute.wait_stream()` and the additional memory pressure from double buffered pinned allocations, exceeds the potential savings and produces a net regression. Notably, the measured H2D time for CUDA Streams (2.02 ms)



TABLE I  
PERFORMANCE SUMMARY ACROSS ALL CONFIGURATIONS

| Configuration                       | Speedup<br>(vs base) | Throughput<br>(samp/s) | Epoch Time<br>(s) | Peak VRAM<br>(MB) | Best mIoU |
|-------------------------------------|----------------------|------------------------|-------------------|-------------------|-----------|
| Baseline (FP32 Eager)               | 1.00×                | 7.99                   | 128.4             | 12,802            | 0.5395    |
| AMP (Mixed Precision)               | 4.92×                | 39.35                  | 25.9              | 6,999             | 0.5370    |
| <code>torch.compile</code> (Fusion) | 1.33×                | 10.65                  | 97.5              | 12,445            | 0.5301    |
| CUDA Streams (Async)                | 0.86×                | 6.89                   | 148.8             | 12,844            | 0.5395    |
| AMP + <code>torch.compile</code>    | 5.27×                | 42.09                  | 24.4              | 6,594             | 0.5313    |

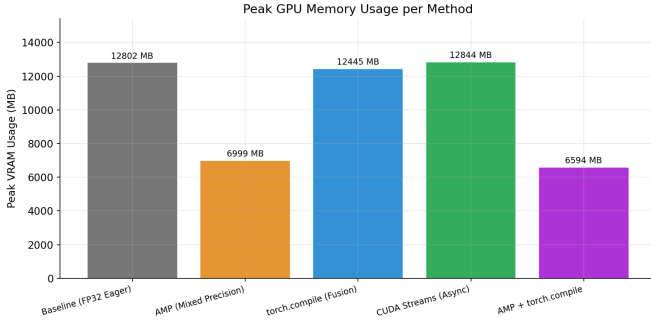


Fig. 3. Peak VRAM usage per configuration. AMP based configurations reduce memory consumption by approximately 45% relative to baseline, from 12,802 MB to under 7,000 MB.

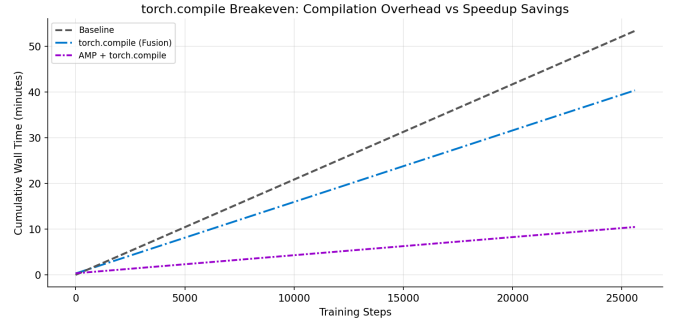


Fig. 4. `torch.compile` breakeven analysis showing cumulative wall time as a function of training steps. Both compiled configurations recover their one time compilation cost within the first epoch.

is slightly higher than the baseline (1.74 ms), consistent with increased contention from the double buffer allocation pattern.

#### D. Memory Efficiency

Figure 3 shows peak VRAM usage per configuration. AMP reduces peak memory from 12,802 MB to 6,999 MB, a reduction of approximately 45%, by storing activations and weights in FP16 rather than FP32. This is the most practically significant secondary benefit of AMP: on the RTX 5070 Ti’s 16 GB GDDR7 frame buffer, the baseline configuration consumes approximately 80% of available VRAM at batch size 8, leaving limited headroom for larger batches or higher resolution inputs. AMP + `torch.compile` achieves the lowest peak memory at 6,594 MB, as `torch.compile`’s elimination of intermediate tensor materialization further reduces the activation memory footprint. `torch.compile` alone and CUDA Streams alone show no meaningful memory reduction relative to baseline, as neither method changes the numerical precision of stored tensors.

#### E. Compilation Cost and Amortization

Figure 4 shows the cumulative wall time for baseline, `torch.compile`, and AMP + `torch.compile` as a function of training steps. `torch.compile` incurs an 18-second one time compilation cost, and AMP + `torch.compile` incurs a 20-second cost. Given the per step savings of each configuration relative to baseline, both compiled variants reach breakeven well within the first epoch of training, and the

cumulative wall time advantage grows monotonically thereafter. Over 100 epochs at 4,375 steps per epoch, AMP + `torch.compile` reduces total training wall time from approximately 213 minutes (baseline) to approximately 41 minutes, a saving of approximately 172 minutes attributable entirely to execution level optimization with no modification to model architecture or training procedure.

### VI. QUALITATIVE RESULTS

Figures 5 and 6 show what the model actually predicts on held out validation images compared to the ground truth. Each figure shows the input image, the ground truth mask, and the model prediction side by side. Across both examples the model picks up the dominant terrain correctly. It struggles with sparse classes like big rock, which makes sense given how underrepresented they are in AI4Mars. All five configurations produce basically the same predictions visually, which lines up with the mIoU numbers being so close across methods.

### VII. DISCUSSION AND FUTURE WORK

This paper evaluated three parallelization strategies—AMP, `torch.compile`, and CUDA Streams—for U-Net training on Mars terrain segmentation on a consumer RTX 5070 Ti. The biggest single method gain came from AMP at 4.92×, which makes sense because the baseline is not even using the Tensor Cores at all, so enabling AMP is basically unlocking hardware that was just sitting there. The CUDA Streams result going backwards at 0.86× was the most interesting finding. The H2D transfer is less than 1 ms against over 1,000 ms of

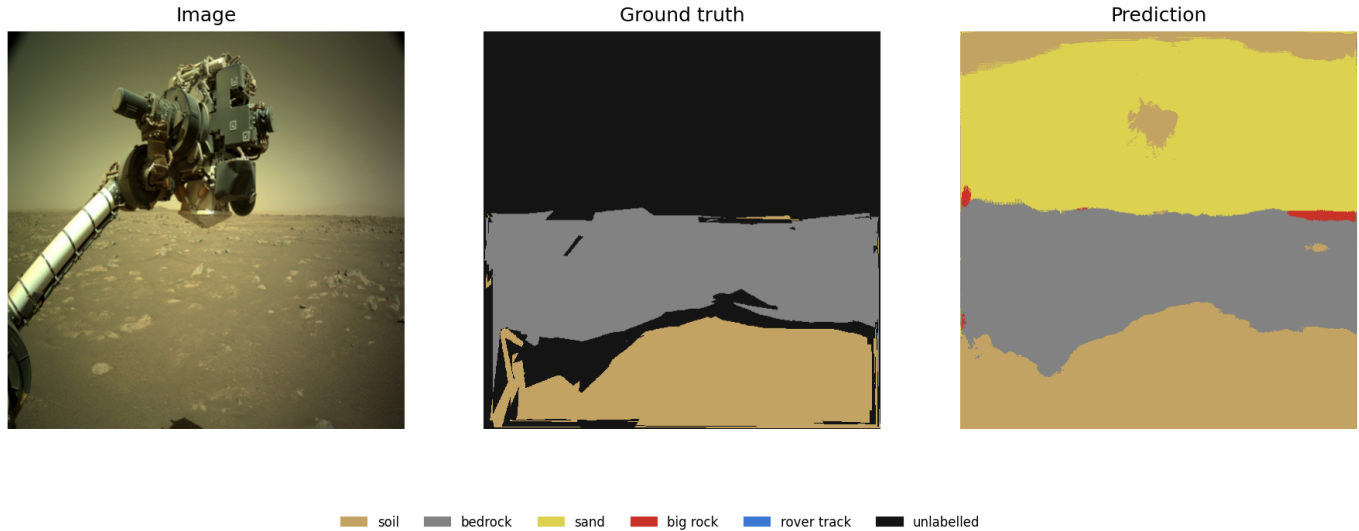


Fig. 5. Inference example 1: rover arm scene. The model gets the bedrock band and sand region right in the lower half. The sky area is unlabelled in the ground truth but the model predicts it as sand, probably because the pale Martian sky looks similar to sandy terrain at this resolution.

compute, so there is almost nothing to overlap and the stream management overhead ends up costing more than it saves. All accuracy differences stayed under 0.6% mIoU so none of these methods are actually hurting the model.

The AMP + `torch.compile` combination was not in the original plan, which was just the three individual methods. After seeing how AMP introduced dtype cast overhead at each skip connection boundary and knowing that `torch.compile` specifically targets redundant kernel launches, it seemed worth testing whether the compiler could clean up the overhead that AMP adds. It did — the  $5.27\times$  result is better than either method alone and the reason is that the compiler fuses away exactly the cast penalty that was limiting AMP’s standalone performance. The takeaway is that these methods are not mutually exclusive and testing combinations is worth it, especially when one method’s overhead lines up with what another method targets.

There are a few things this study does not cover. All experiments ran on a single GPU so there is no data on how these optimizations hold up in a multi GPU setup. CUDA Streams was also never tested with AMP together, so whether AMP’s reduced H2D volume would be enough to let prefetching actually work is still an open question. And since only mean mIoU was reported it is hard to say how individual rare classes like rover tracks performed across configurations.

Future work should investigate: (1) combining all three methods with a higher throughput CPU DataLoader (e.g., NVIDIA DALI) to relieve the CPU bottleneck and allow CUDA Streams to realize its theoretical overlap; (2) extending the analysis to multi GPU training, with a focus on the computational challenges that arise at scale—specifically, how gradi-

ent compression, overlapping AllReduce communication with the backward pass, and applying AMP and `torch.compile` within `torch.distributed` affect both throughput and convergence when gradient traffic across GPUs becomes the dominant bottleneck; and (3) applying the same optimization framework to transformer based segmentation architectures such as SegFormer, where the attention mechanism’s memory access pattern may alter the relative benefit of kernel fusion versus mixed precision.

## REFERENCES

- [1] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional networks for biomedical image segmentation,” in *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, 2015, pp. 234–241.
- [2] P. Micikevicius et al., “Mixed precision training,” in *Int. Conf. on Learning Representations (ICLR)*, 2018.
- [3] J. Ansel et al., “PyTorch 2: Faster machine learning through dynamic Python bytecode transformation and graph compilation,” in *Proc. 29th ACM Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2024, pp. 929–947. doi: [10.1145/3620665.3640366](https://doi.org/10.1145/3620665.3640366)
- [4] R. M. Swan et al., “AI4MARS: A dataset for terrain-aware autonomous driving on Mars,” in *Proc. IEEE/CVF Conf. on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2021, pp. 1982–1991.
- [5] NVIDIA Corporation, “CUDA C++ Programming Guide,” Version 12.x, 2024. [Online]. Available: <https://docs.nvidia.com/cuda/cuda-c-programming-guide>
- [6] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *Int. Conf. on Learning Representations (ICLR)*, 2019.
- [7] C. H. Sudre et al., “Generalised Dice overlap as a deep learning loss function for highly unbalanced segmentations,” in *Deep Learning in Medical Image Analysis*, 2017.
- [8] G. M. Amdahl, “Validity of the single processor approach to achieving large scale computing capabilities,” in *Proc. AFIPS Spring Joint Computer Conference*, 1967, pp. 483–485.

NLF\_0102\_0676000653\_617ECM\_N0040372NCAM03101\_07\_195J

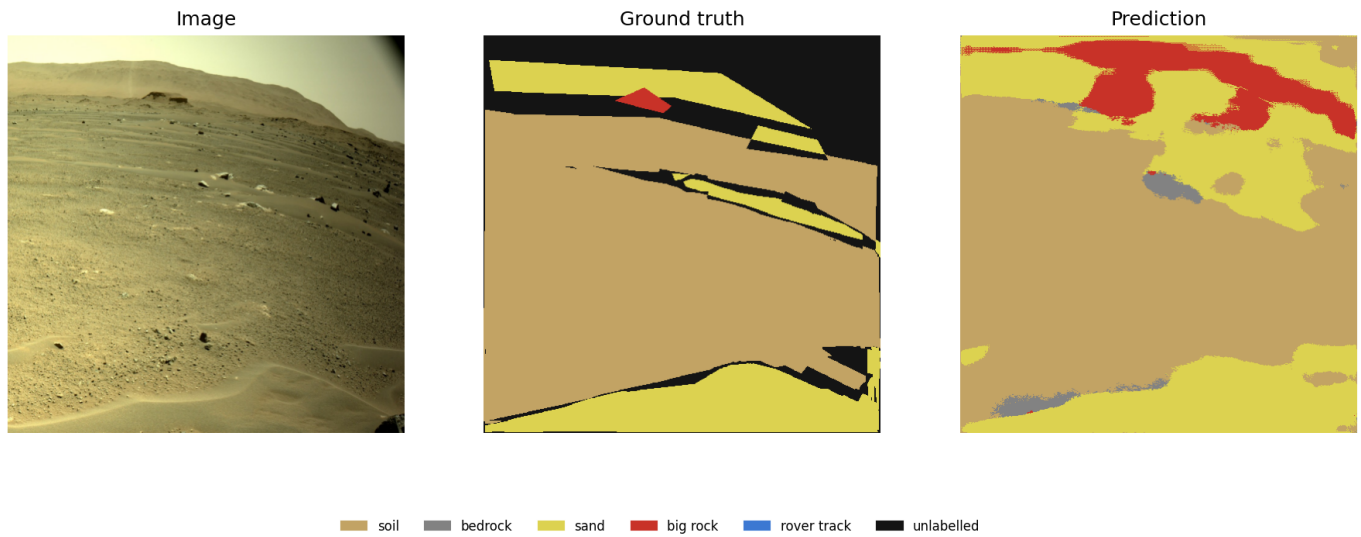


Fig. 6. Inference example 2: open dune landscape. The model gets the general soil to sand split and picks up the big rock region. It does over extend big rock into the upper sky area, which is a good example of how hard it is for the model to tell apart visually similar unlabelled regions from rare foreground classes.